

Payoff

Less code duplication (if the data handling code is put in a central place)

Better code organization (methods for handling data are next to the actual data)

When to Ignore

Sometimes behavior is purposefully kept separate from the class that holds the data. The usual advantage of this is the ability to dynamically change the behavior (see Strategy, Visitor and other patterns)

Feature Envy (حسادت به ویژگی)

علائم و نشانه‌ها (Signs and Symptoms)

یه متد بیشتر به داده‌های یه شیء دیگه دسترسی داره تا به داده‌های خودش.

دلایل ایجاد (Reasons for the Problem)

این بو ممکنه بعد از انتقال فیلدها به یه Data Class اتفاق بیفته. اگه اینطوره، باید عملیات روی داده‌ها رو هم به همون کلاس منتقل کنی.

درمان (Treatment)

#	تکنیک	کاربرد
۱	Move Method	اگه کل متد باید به جای دیگه منتقل بشه
۲	Extract Method + Move Method	اگه فقط بخشی از متد به داده‌های شیء دیگه دسترسی داره

کاربرد	تکنیک	#
اگره متد از چند کلاس استفاده می‌کنه، بین کدوم کلاس بیشترین داده رو داره و متد رو ببر اونجا	تشخیص کلاس اصلی	۳

مثال

csharp

```
// Feature Envy ❌: این متد بیشتر به داده‌های Customer دسترسی داره تا خودش
public class Invoice
{
    public Customer Customer { get; set; }
    public List<Item> Items { get; set; }

    ()public string GetCustomerInfo
    {
        // همه چی از Customer میاد!
        return $"{Customer.Name} - {Customer.Email} - {Customer.Phone} - {Customer.Address}";
    }
}
```

بعد از Move Method:

csharp

```
public class Customer
{
    public string Name { get; set; }
    public string Email { get; set; }
    public string Phone { get; set; }
    public string Address { get; set; }

    ()public string GetFullInfo
    {
        return $"{Name} - {Email} - {Phone} - {Address}";
    }
}
```

```

        public class Invoice
        {

        public Customer Customer { get; set; }
        public List<Item> Items { get; set; }

        {

```

Payoff (دستاورد)

- کد تکراری کمتر (کد مدیریت داده یه جا جمع میشه)
- سازماندهی بهتر (متدهای مربوط به داده کنار خود داده هستن)

When to Ignore

گاهی رفتار عمداً از کلاس داده جدا نگه داشته میشه. مزیتش: توانایی تغییر پویای رفتار (مثل الگوهای Visitor و Strategy).

اصل طلایی

چیزهایی که با هم تغییر می‌کنن، باید یه جا باشن. داده و توابعی که ازش استفاده می‌کنن معمولاً با هم تغییر می‌کنن.

تمرین

این کد رو با درمان Feature Envy اصلاح کن:

csharp

```

        public class Product
        {

        public string Name { get; set; }
        public decimal Price { get; set; }
        public decimal Weight { get; set; }

```

```
{  
  
    public class ShippingCalculator  
    {  
  
        public decimal CalculateShipping(Product product)  
        {  
            // بیشتر با داده‌های Product کار می‌کنه تا خودش!  
            if (product.Weight < 1)  
                ;return 5000  
            if (product.Weight < 5)  
                ;return 10000  
            if (product.Weight < 20)  
                ;return 25000  
                ;return 50000  
        }  
    }  
}
```

راهنمایی: CalculateShipping رو به Product منتقل کن.

بفرست تا بررسی کنیم! 🍷